

Adaptive Page Replacement Based on Memory-Reference Locality

An Experimental Evaluation of FIFO, LRU, Optimal, Clock, LFU, and an Online
Adaptive Policy

Course: BCSE303L – Operating Systems

Semester: Fall Semester 2026-27

Case Study: CS3 – Adaptive Page Replacement

Students:

24BCE0702 – Yash Pradhan

24BCE0714 – Anjini Pandey

Date: October 2026

Declaration / Certificate

We hereby declare that this project report titled "*Adaptive Page Replacement Based on Memory-Reference Locality*" is an original piece of work carried out by us for the course BCSE303L – Operating Systems during the Fall Semester 2026-27. The project was designed, implemented, tested, and evaluated as part of our coursework, with all reported numerical results generated from the project's experimental implementation. No results, plots, or data have been fabricated. All numerical values presented in this report are sourced directly from the project's experimental CSV output files.

Student 1	24BCE0702 – Yash Pradhan
Student 2	24BCE0714 – Anjini Pandey
Faculty / Guide	_____
Date	_____

Abstract

Virtual memory systems rely on page replacement policies to determine which resident page to evict when physical memory is saturated and a page fault occurs. Classic algorithms such as LRU, LFU, FIFO, and Clock embed fixed assumptions about program behavior and suffer systemic degradation when workload access patterns shift. This case study designs, implements, and rigorously evaluates an online, strictly causal **Adaptive Page Replacement** policy that dynamically detects memory-reference locality within a rolling observation window and switches between LRU, Clock, and LFU replacement heuristics without any future lookahead.

We implemented five baseline algorithms (FIFO, LRU, Optimal/Belady's MIN, Clock/Second-Chance, and LFU with FIFO tie-breaking) and two adaptive configurations (W=50 and W=100). The complete experimental evaluation comprised **1,260 raw experimental runs** across **252 unique configurations**: 4 workload types × 3 workload sizes × 3 frame capacities × 7 algorithm configurations, each with 5 timing repetitions.

Results demonstrate that the adaptive engine successfully detects sequential scan patterns (performing 312 total policy switches on N=20,000 sequential workloads with W=50 (156 transitions to Clock and 156 transitions back to LRU)) and preserves physical frames across policy transitions without cold-start penalties. However, the fixed heuristic thresholds were misaligned with the workload statistics: Adaptive W50 matched LRU's fault count in 30 of 36 conditions, while LFU had fewer faults than both adaptive variants in all nine high-locality configurations. At the representative N = 5,000, C = 5 condition, LFU achieved 38.34% hit ratio versus 30.84% for each adaptive configuration. We conclude that adaptive page replacement is architecturally viable but that threshold calibration is the critical determinant of practical effectiveness.

Keywords

page replacement, virtual memory, operating systems, memory locality, FIFO, LRU, LFU, Clock, second chance, optimal replacement, Belady's MIN, adaptive replacement, working set, page faults, hit ratio, Belady's anomaly, sliding window, coefficient of variation

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Background and Operating-System Theory
5. Page Replacement Algorithms
6. System Architecture
7. Project Directory Structure
8. Detailed Implementation
9. Adaptive Page Replacement Design
10. Workload Generation
11. Experimental Methodology
12. Textbook Validation
13. Unit Testing
14. Experimental Matrix
15. Page Fault Results
16. Hit Ratio Results
17. Execution-Time Results
18. Adaptive Behavior Analysis
19. Belady's Anomaly
20. Statistical Analysis
21. Comparative Discussion
22. Limitations
23. Individual Contributions
24. Reproducibility
25. Conclusion
26. Future Work
27. References

1. Introduction

With demand paging, a process can execute while only a subset of its pages resides in physical memory frames. When a requested page is not in physical memory, a **page fault** occurs, requiring the operating system to load the page from secondary storage. If all physical frames are occupied, the OS must select a resident page to evict. Because secondary storage access is orders of magnitude slower than RAM access, the choice of **page replacement policy** is a primary determinant of system throughput and responsiveness.

Classic replacement policies embed fixed assumptions about future reference behavior: FIFO assumes old pages are obsolete, LRU assumes recent accesses predict future reuse, and LFU assumes popularity determines importance. In practice, programs exhibit **dynamic phases** of execution — tight iterative loops, sequential data scans, and random accesses. No single fixed assumption holds true across all phases.

The motivation for **adaptive page replacement** is to create a responsive OS mechanism that dynamically shifts its replacement heuristic based on the statistical properties of the currently observed reference stream. This case study explores this through rigorous simulation and statistical evaluation.

2. Problem Statement

Research Question: "Can an adaptive page-replacement policy select an appropriate strategy according to observed memory-reference locality?"

Fixed page replacement policies assume monolithic program behavior, causing elevated fault rates when workloads change character. Adaptive selection aims to improve performance by matching the replacement heuristic to the current locality phase. However, dynamic selection introduces challenges: correctly identifying the workload phase using only online causal history, managing state during policy transitions, and controlling the computational overhead of metric tracking.

3. Objectives

1. Implement FIFO page replacement with strict queue-based eviction.
2. Implement LRU page replacement using `OrderedDict` for $O(1)$ recency tracking.
3. Implement Optimal (Belady's MIN) with precomputed next-occurrence queues.
4. Implement Clock (Second-Chance) with circular buffer and reference bits.
5. Implement LFU with frequency tracking and deterministic FIFO tie-breaking.

6. Validate all five algorithms against a standard textbook reference trace.
7. Generate four distinct reproducible synthetic workload patterns.
8. Develop a causal adaptive mechanism based on rolling observation windows.
9. Measure page faults, hit ratios, and execution time across all policies.
10. Compare adaptive and fixed policies across 252 experimental configurations.
11. Generate nine publication-grade statistical plots.
12. Produce comprehensive statistical summaries and a technical report.

4. Background and Operating-System Theory

4.1 Virtual Memory and Demand Paging

Virtual memory allows processes to use logical addresses mapped to physical frames by the MMU. Demand paging loads pages only when referenced, reducing initial memory requirements. When a page fault occurs and physical memory is full, the OS invokes the page replacement algorithm to select a victim page.

4.2 Locality of Reference

Programs exhibit predictable memory access patterns: **temporal locality** (recently accessed pages are likely accessed again) and **spatial locality** (nearby pages are likely accessed soon). A third characteristic — **diversity** — describes the ratio of unique pages accessed in a given time window.

4.3 The Working Set Model

Denning's Working Set Model (1968) defines the working set as the set of pages referenced in a recent time window. If physical frames are fewer than the working set size, the process experiences **thrashing**.

4.4 Belady's Anomaly

For FIFO, increasing physical frames can paradoxically **increase** page faults. Algorithms satisfying the **stack property** (LRU, Optimal) are immune to this anomaly.

5. Page Replacement Algorithms

5.1 FIFO (First-In, First-Out)

Evicts the page that has resided in memory the longest. Uses a `deque` for insertion order and a `set` for $O(1)$ residency checks. Simple but ignores locality; susceptible to Belady's Anomaly.

5.2 LRU (Least Recently Used)

Evicts the page whose most recent access is oldest. Uses `OrderedDict` with `move_to_end()` for $O(1)$ recency promotion. Highly effective for temporal locality but fails on sequential scans.

5.3 Optimal (Belady's MIN)

Evicts the page whose next access is farthest in the future. Requires complete future knowledge — serves as an offline theoretical lower bound only.

5.4 Clock (Second-Chance)

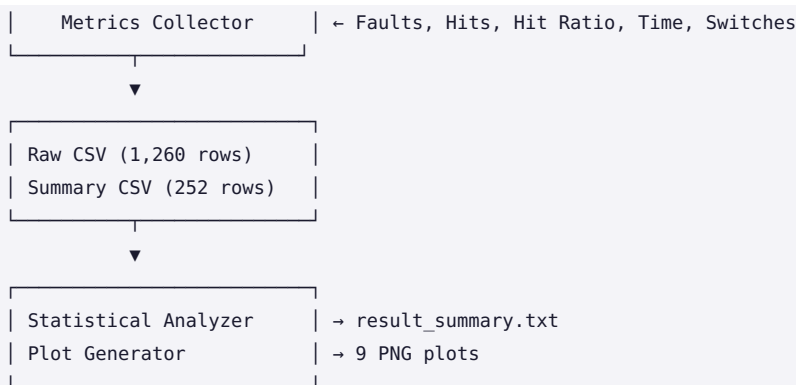
Approximates LRU with a circular buffer and reference bits. On eviction, the hand sweeps clearing bits until finding a victim (`ref_bit=0`). In our implementation, the hand does NOT advance on hits, producing 14 faults on the textbook string.

5.5 LFU (Least Frequently Used)

Evicts the page with the lowest reference frequency. Ties are broken by FIFO arrival order (oldest arrival evicted first). Excels at protecting "hot" pages in skewed distributions.

6. System Architecture





Adaptive Engine Internal Flow

Incoming Reference

- Rolling Window (deque, W=50 or 100)
- Locality Metrics ($U = \text{unique}/W$, $CV = \text{std}/\text{mean}$)
- Decision Rules ($U < 0.35 \rightarrow \text{LRU}$, $U \geq 0.85 \rightarrow \text{Clock}$, $CV \geq 1.0 \rightarrow \text{LFU}$)
- State Handover (if strategy changed)
- Execute Replacement
- Append to Window

7. Project Directory Structure

```
CS3-Adaptive-Page-Replacement/
├── config/config.json          # Parameters, thresholds, seeds
├── src/
│   ├── common.py, fifo.py, lru.py, optimal.py, clock.py, lfu.py, adaptive.py
├── workload/
│   ├── workload_generator.py    # Seed=42 trace synthesizer
│   └── traces/                  # 12 JSON trace files
├── experiments/
│   ├── run_validation.py, run_experiments.py, analyze_results.py
├── tests/
│   ├── test_fifo.py, test_lru.py, test_optimal.py, test_clock.py, test_lfu.py, test_adaptive.py
├── results/
│   ├── raw_results.csv (1,260 rows), summary_results.csv (252 rows)
│   └── validation_results.csv, belady_anomaly.csv
├── plots/                      # 9 PNG figures
├── report/                     # Report files + setup telemetry
└── CONTRIBUTIONS.md, README.md, requirements.txt
```

8. Detailed Implementation

8.1 FIFO Implementation

```
# Listing 1 – FIFO replacement logic (src/fifo.py)
queue = deque()
resident_set: Set[int] = set()

for page in reference_string:
    if page in resident_set:
        page_hits += 1
    else:
        page_faults += 1
        if len(queue) == frame_capacity:
            evicted_page = queue.popleft()
            resident_set.remove(evicted_page)
        queue.append(page)
        resident_set.add(page)
```

8.2 LRU Implementation

```
# Listing 2 – LRU update and eviction logic (src/lru.py)
resident_frames = OrderedDict()

for page in reference_string:
    if page in resident_frames:
        page_hits += 1
        resident_frames.move_to_end(page) # promote to MRU
    else:
        page_faults += 1
        if len(resident_frames) == frame_capacity:
            resident_frames.popitem(last=False) # evict LRU
        resident_frames[page] = True
```

8.3 Optimal Implementation

```
# Listing 3 – Optimal farthest-future lookahead (src/optimal.py)
for resident_page in resident_set:
    occ = future_occurrences[resident_page]
    if not occ:
        victim_page = resident_page # never used again
        break
    else:
        next_use = occ[0]
        if next_use > farthest_idx:
```

```
farthest_idx = next_use
victim_page = resident_page
```

8.4 Clock Implementation

```
# Listing 4 – Clock reference-bit handling (src/clock.py)
while True:
    if frames[hand]["ref_bit"] == 0:
        frames[hand] = {"page": page, "ref_bit": 1}
        hand = (hand + 1) % frame_capacity
        break
    else:
        frames[hand]["ref_bit"] = 0 # second chance
        hand = (hand + 1) % frame_capacity
```

8.5 LFU Implementation

```
# Listing 5 – LFU tie-breaking and eviction (src/lfu.py)
victim = min(
    resident.keys(),
    key=lambda p: (resident[p]["freq"], resident[p]["arrival_order"])
)
del resident[victim]
resident[page] = {"freq": 1, "arrival_order": step}
```

9. Adaptive Page Replacement Design

9.1 Rolling Window

A `collections.deque(maxlen=W)` stores the last W page references. The engine evaluates metrics every $\max(10, W/2)$ steps once the window is full.

9.2 Unique Page Ratio (U)

$U = |\text{unique pages in } W| / W$. Low U (<0.35) $\rightarrow$ temporal locality. High U (≥ 0.85) $\rightarrow$ sequential scan.

9.3 Frequency CV

$CV_f = \sigma_f / \mu_f$ where f represents per-page frequencies within W . High CV (≥ 1.0) $\rightarrow$ skewed popularity.

9.4 Decision Rules

```
# Listing 8 – Adaptive strategy selection (src/adaptive.py)
if u_ratio < 0.35:          selected = "LRU"
elif u_ratio >= 0.85:       selected = "Clock"
elif cv_f >= 1.0 and u < 0.7: selected = "LFU"
else:                      selected = "LRU"
```

Condition	Interpretation	Selected Policy
$U < 0.35$	Strong temporal locality	LRU
$U \geq 0.85$	Sequential scan / high diversity	Clock
$CV \geq 1.0$ and $U < 0.70$	Skewed frequency	LFU
Otherwise	No strong signal	LRU (default)

9.5 State Handover

Switch to LRU: Resident pages ordered by last occurrence in history window.

Switch to LFU: Frequencies initialized from window `Counter`.

Switch to Clock: All `ref_bits` set to 1; hand reset to 0.

9.6 Causality Guarantee

The decision logic runs *before* the current reference is appended to the window. The `test_causality_future_independence` unit test mutates future references and verifies identical decisions up to the mutation point.

10. Workload Generation

Workload	Generator Logic	Expected Behavior	Purpose
High Locality	80% refs from 10 hot pages, 20% from 40 cold pages	Skewed Pareto 80/20	Test LRU/LFU on hot sets
Sequential	Cyclic scan 0 – 49, 2% random jump	Streaming scan	Test scan resistance
Random	Uniform over 50 pages	Weak locality	Baseline performance
Looping	Fixed cycle 0,1,2,3,4,5	Deterministic loop (size 6)	Test thrashing vs capacity

```
# Listing 6 – High Locality generator (workload/workload_generator.py)
for _ in range(n):
    if rng.random() < 0.8:
        refs.append(rng.choice(hot_pages))
    else:
        refs.append(rng.choice(cold_pages))
```

11. Experimental Methodology

Parameter	Values
Random Seed	42
Workload Types	High Locality, Sequential, Random, Looping
Workload Sizes (N)	1,000; 5,000; 20,000
Frame Capacities (C)	3, 5, 8
Algorithms	FIFO, LRU, Optimal, Clock, LFU, Adaptive W50, Adaptive W100
Timing Repetitions	5 per configuration
Timing Method	<code>time.perf_counter()</code>

Hardware / Software Environment

Component	Specification
CPU	Intel Core Ultra 7 155H (16 cores / 22 threads, 3800 MHz)
RAM	16 GB (15.46 GiB usable)
OS	Windows 11 Home Single Language, 64-bit (Build 10.0.26200)
Python	CPython 3.13.4 (MSC v.1943, 64-bit)
matplotlib	3.11.2
numpy	2.5.3

12. Textbook Validation

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Frames: 3

Algorithm	Faults	Hits	Hit Ratio	Expected	Status
FIFO	15	5	25.00%	15	PASS
LRU	12	8	40.00%	12	PASS
Optimal	9	11	55.00%	9	PASS
Clock	14	6	30.00%	14	PASS
LFU	13	7	35.00%	13	PASS

Clock (14) and LFU (13) values match the documented deterministic implementation semantics (hand-advance-on-fill for Clock; FIFO arrival-order tie-breaking for LFU).

13. Unit Testing

29 unit tests across 6 test files. All pass: Ran 29 tests in 0.009s – OK

Test File	Tests	Key Assertions
test_fifo.py	5	Textbook faults=15; edge cases
test_lru.py	4	Textbook faults=12; looping thrash
test_optimal.py	3	Textbook faults=9; lower-bound property
test_clock.py	4	Textbook faults=14; ref-bit semantics
test_lfu.py	4	Textbook faults=13; FIFO tie-breaking
test_adaptive.py	9	Invariants; causality future-independence; scan → Clock; locality → LRU
Total	29	

14. Experimental Matrix

4 workloads × 3 sizes × 3 capacities × 7 policies = **252 configurations**

252 × 5 repetitions = **1,260 raw experimental runs**

15. Page Fault Results

Workload	N	C	FIFO	LRU	Optimal	Clock	LFU	Adap. W50	Adap. W100
high_locality	5,000	5	3,495	3,458	2,167	3,494	3,083	3,458	3,458
high_locality	20,000	5	14,036	13,812	8,711	13,992	12,373	13,809	13,812
sequential	5,000	5	4,964	4,962	4,431	4,964	4,650	4,962	4,962
random	5,000	5	4,505	4,502	3,409	4,505	4,519	4,502	4,502
looping	5,000	5	5,000	5,000	1,004	5,000	5,000	5,000	5,000
looping	5,000	8	6	6	6	6	6	6	6

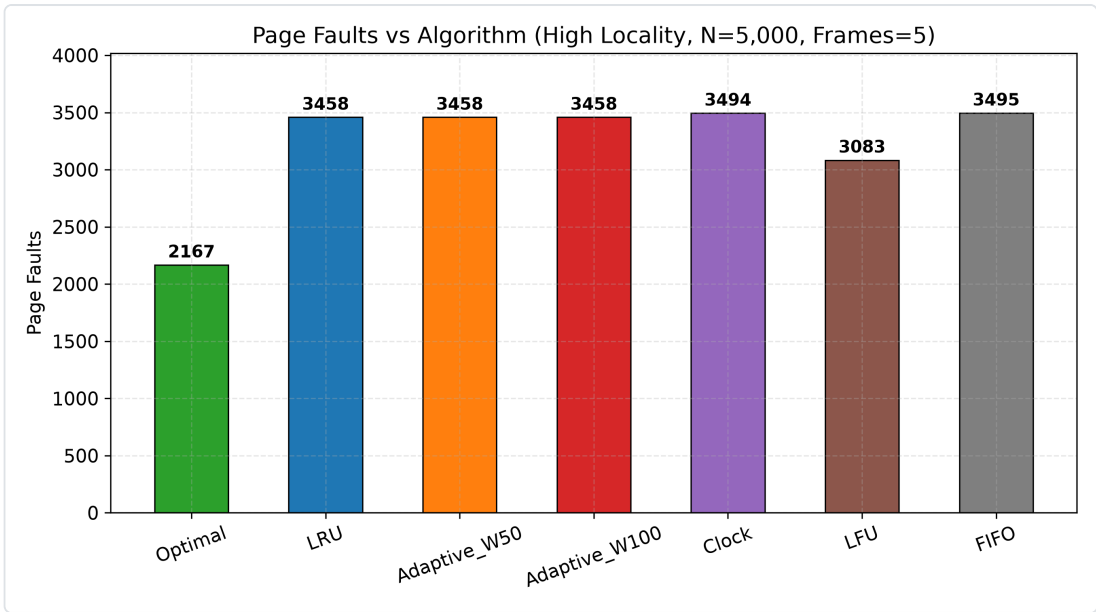


Figure 1: Page Faults vs Algorithm (High Locality, N=5,000, Frames=5). LFU achieves fewest faults (3,083) among online policies.

16. Hit Ratio Results

Workload	FIFO	LRU	Clock	LFU	Adap. W50	Adap. W100	Optimal
high_locality (N=5k, C=5)	30.10%	30.84%	30.12%	38.34%	30.84%	30.84%	56.66%
sequential	0.72%	0.76%	0.72%	7.00%	0.76%	0.76%	11.38%
random	9.90%	9.96%	9.90%	9.62%	9.96%	9.96%	31.82%

looping	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	79.92%
---------	-------	-------	-------	-------	-------	-------	--------

Overall Algorithm Summary (across 36 conditions)

Algorithm	Mean Hit Ratio	Mean Time (ms)
Optimal	43.34%	8.082
LFU	22.87%	15.816
LRU	19.45%	3.507
Adaptive W50	19.46%	18.971
Adaptive W100	19.45%	16.122
Clock	19.26%	9.219
FIFO	19.10%	2.392

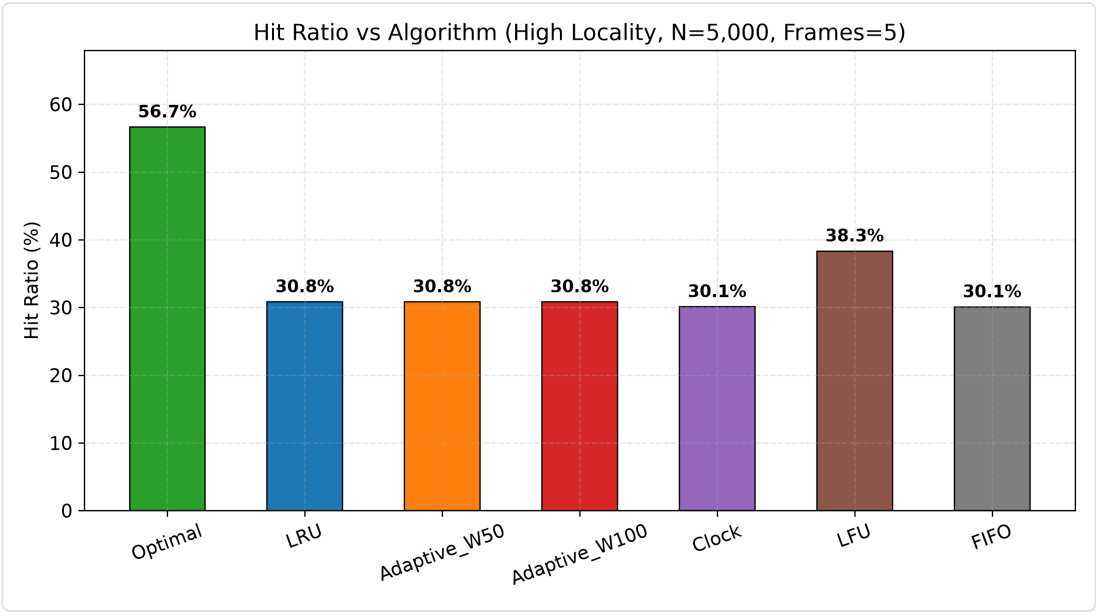


Figure 2: Hit Ratio vs Algorithm (High Locality, N=5,000, Frames=5). LFU dominates at 38.34%.

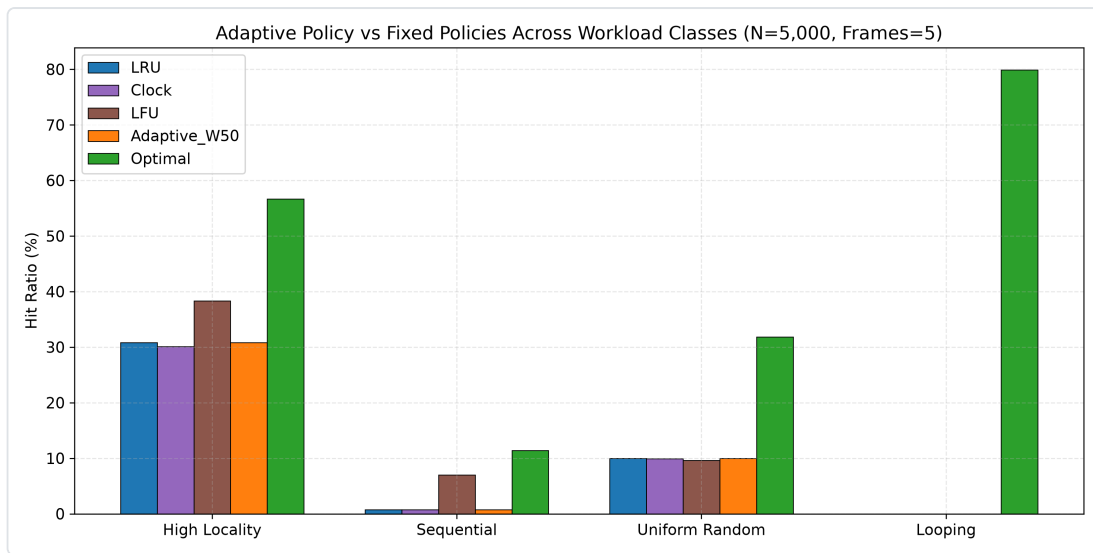


Figure 3: Adaptive vs Fixed Policies across all 4 workload classes (N=5,000, Frames=5).

17. Execution-Time Results

N	FIFO	LRU	Optimal	Clock	LFU	Adap. W50	Adap. W100
1,000	0.27	0.48	1.00	1.12	1.99	2.29	1.79
5,000	1.30	2.02	4.60	5.30	8.87	10.81	9.03
20,000	5.61	8.02	18.65	21.23	36.58	43.82	37.55

Adaptive W50 = 5.4× LRU. Adaptive W100 = 4.6× LRU. FIFO is fastest. Execution time increased approximately linearly with N over the tested sizes; this is an empirical observation rather than a formal asymptotic proof.

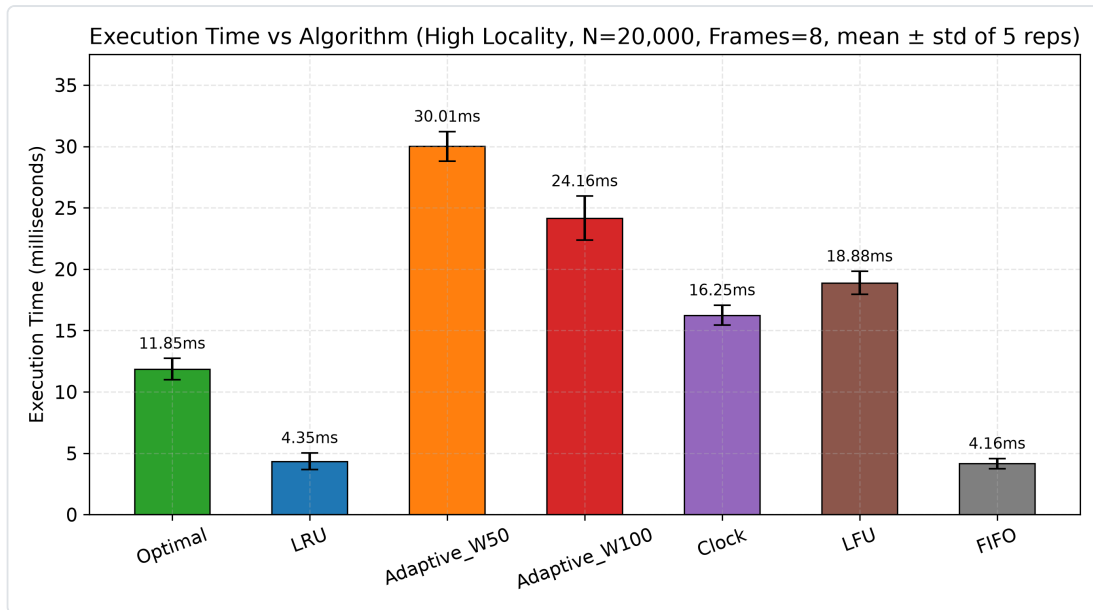


Figure 4: Execution Time vs Algorithm (High Locality, N=20,000, Frames=8). Error bars show std dev across 5 repetitions.

18. Adaptive Behavior Analysis

Workload	W=50 Switches	W=50 Final	W=100 Switches	W=100 Final
High Locality (N=20k, C=5)	2	LRU	0	LRU
Sequential (N=20k, C=5)	312	LRU	0	LRU
Random (N=20k, C=5)	0	LRU	0	LRU
Looping (N=20k, C=5)	0	LRU	0	LRU

Why W=100 never switches: With 50 unique pages, $U_{\max} = 50/100 = 0.50$. The scan threshold (0.85) is mathematically unreachable. For the high-locality workload, the $U < 0.35$ locality condition is evaluated before the LFU condition, so LRU is selected whenever the window is classified as strongly local; therefore, a high CV value cannot trigger LFU while that locality rule is satisfied. Result: Adaptive W100 selected LRU across all 36 W100 configurations (180 timing repetitions).

Adaptive W50 vs LRU: Identical faults in 30/36 conditions; fewer faults in 6 conditions (max reduction: 6 faults); never worse.

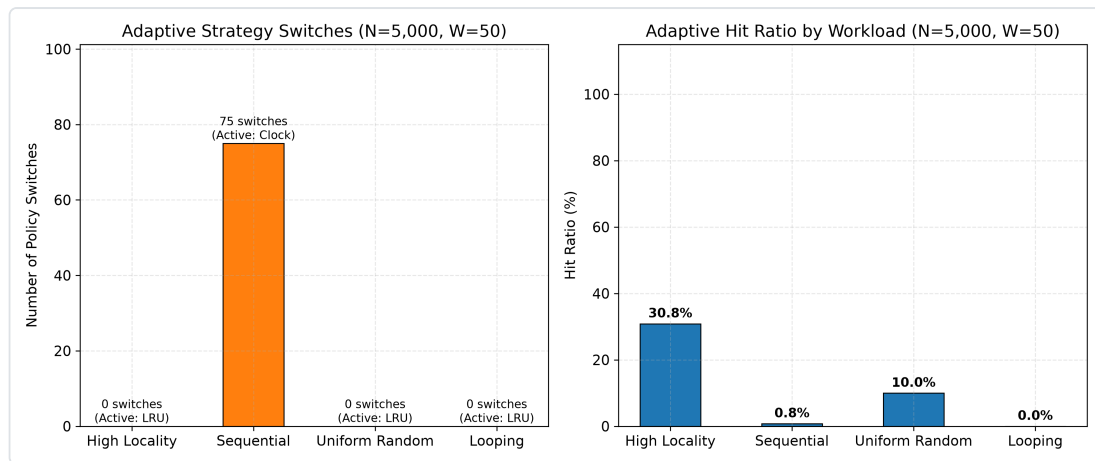


Figure 5: Adaptive Behavior by Workload (N=5,000, W=50). Left: strategy switches. Right: hit ratio achieved.

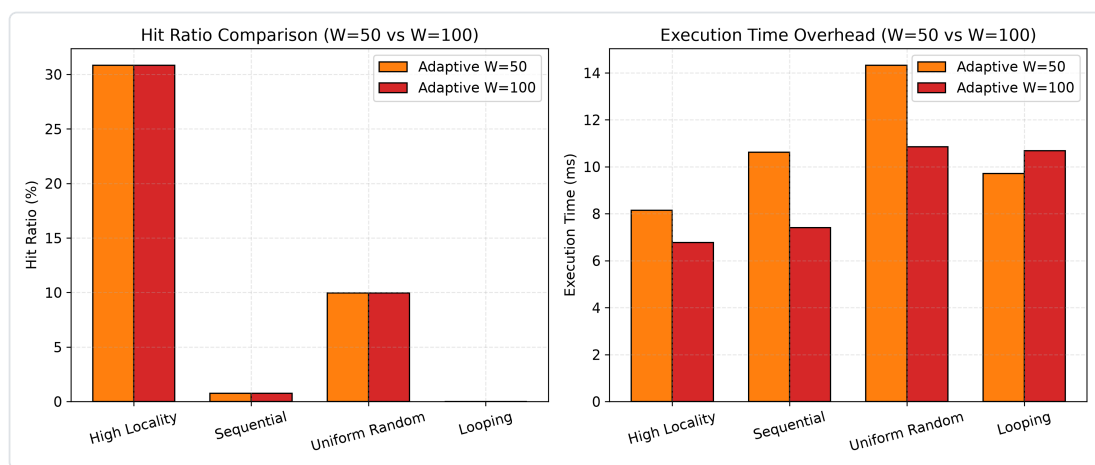


Figure 6: W=50 vs W=100 comparison (N=5,000, Frames=5). Hit ratios are identical; W=50 has higher overhead.

19. Belady's Anomaly

Classic demonstration: FIFO on 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Frames	FIFO Faults	Hit Ratio
3	9	25.00%
4	10	16.67%

4 frames → MORE faults (10) than 3 frames (9). Anomaly confirmed. No anomaly observed in any of the four synthetic N=1,000 workloads across frames 3–8.

20. Statistical Analysis

Fault counts are perfectly deterministic: Standard deviation = 0 across all 5 repetitions for all 252 configurations. No race conditions.

Execution-time variance is low: Generally under 2 ms std dev even for N=20,000.

No formal hypothesis tests were performed because fault counts have zero variance across repetitions.

21. Comparative Discussion

LFU dominates high-locality: 38.34% vs LRU's 30.84% (N=5k, C=5). Frequency counting protects Pareto hot pages effectively.

LRU beats LFU on random: 9.96% vs 9.62%. Under uniform access, no "hot" pages exist to protect.

Adaptive matches LRU: The decision logic checks $U < 0.35$ before evaluating the LFU CV condition, so strongly local windows select LRU before the frequency-skew signal can trigger LFU. This is the most significant missed opportunity in the current rule ordering.

LFU is among the winners most often: With ties counted for every policy sharing the minimum, LFU is tied for the minimum fault count in 30/36 configurations. The corresponding counts are FIFO 14/36, LRU 14/36, Adaptive W50 14/36, Adaptive W100 14/36, and Clock 11/36.

22. Limitations

1. **Heuristic Thresholds and Rule Ordering:** Manually selected; the current decision ordering can favor LRU before the LFU frequency-skew condition is evaluated, limiting LFU selection on high-locality workloads.
2. **Simulation:** Python interpreter overhead, not kernel-level costs.
3. **Limited Workloads:** 4 synthetic types only; no real-world traces.
4. **Limited Capacities:** Only C=3,5,8 tested.
5. **No Frequency Aging:** LFU lacks decay for stale counts.
6. **Adaptive Overhead:** 5.4× LRU for W=50.
7. **No ARC Comparison:** Advanced scan-resistant baseline not included.
8. **W-U Coupling:** Window size constrains U range (W=100 limits $U_{\max}$ to 0.50).

23. Individual Contributions

Area	24BCE0702 – Yash Pradhan	24BCE0714 – Anjini Pandey
Algorithms	FIFO, LRU, Optimal	Clock, LFU, Adaptive
Testing	FIFO/LRU/Optimal tests	Clock/LFU/Adaptive tests; causality test
Workloads	Workload generator	—
Experiments	—	Batch runner; Belady scan; analysis; 9 plots
Adaptive Design	—	Engine architecture; metrics; state handover

24. Reproducibility

```
# Python 3.10+ required
pip install -r requirements.txt

# Step 1: Unit tests (29 tests)
python -m unittest discover -s tests -v

# Step 2: Textbook validation
python experiments/run_validation.py

# Step 3: Generate workload traces
python workload/workload_generator.py

# Step 4: Full experiment matrix (1,260 runs)
python experiments/run_experiments.py

# Step 5: Analysis and plots
python experiments/analyze_results.py
```

25. Conclusion

This case study empirically evaluates whether an adaptive page-replacement policy can select appropriate strategies according to observed memory-reference locality.

What was implemented: Five classic algorithms and a novel online, causal adaptive engine using U and CV metrics with non-destructive state handover.

What experiments showed: The adaptive engine detects scan patterns and performed 312 total policy switches on the $N = 20,000$ sequential workload with $W = 50$, consisting of 156 transitions to Clock and 156 transitions back to LRU. It matches LRU in 30/36 conditions (W50) and 36/36 (W100).

Where fixed policies are stronger: LFU has fewer faults than both Adaptive variants in all nine high-locality configurations. At the representative $N = 5,000$, $C = 5$ condition, LFU achieved a 38.34% hit ratio versus 30.84% for each adaptive configuration.

Overall: Adaptive page replacement is *architecturally viable* — causality is maintained, state handover works — but *threshold calibration is the critical determinant* of practical effectiveness. The answer to the research question is **yes, with nuance**: adaptive policies can select strategies according to locality, but do not necessarily outperform well-chosen fixed policies without precise threshold tuning.

26. Future Work

1. Dynamic threshold tuning (reinforcement learning / online gradient descent).
2. Better phase-change detection (stack distance, entropy, working-set estimation).
3. More locality features (spatial locality, temporal autocorrelation).
4. Real-world memory traces (SPEC CPU, database workloads).
5. More frame capacities (16, 32, 64+).
6. Cost-aware switching (minimum improvement prediction).
7. Linux kernel module prototype.
8. ARC comparison baseline.
9. Frequency aging for LFU.

27. References

1. Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*, 10th ed., Wiley, 2018.
2. Belady, L. A., "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, 1966.
3. Belady, L. A., Nelson, R. A., Shedler, G. S., "An anomaly in space-time characteristics of certain programs running in a paging machine," *CACM*, vol. 12, no. 6, 1969.
4. Denning, P. J., "The working set model for program behavior," *CACM*, vol. 11, no. 5, 1968.
5. Megiddo, N., Modha, D. S., "ARC: A self-tuning, low overhead replacement cache," *USENIX FAST*, 2003.
6. Corbato, F. J., "A paging experiment with the Multics system," *MIT Project MAC*, 1968.
7. Tanenbaum, A. S., Bos, H. *Modern Operating Systems*, 4th ed., Pearson, 2014.

28. Appendices

Appendix E: Additional Plots

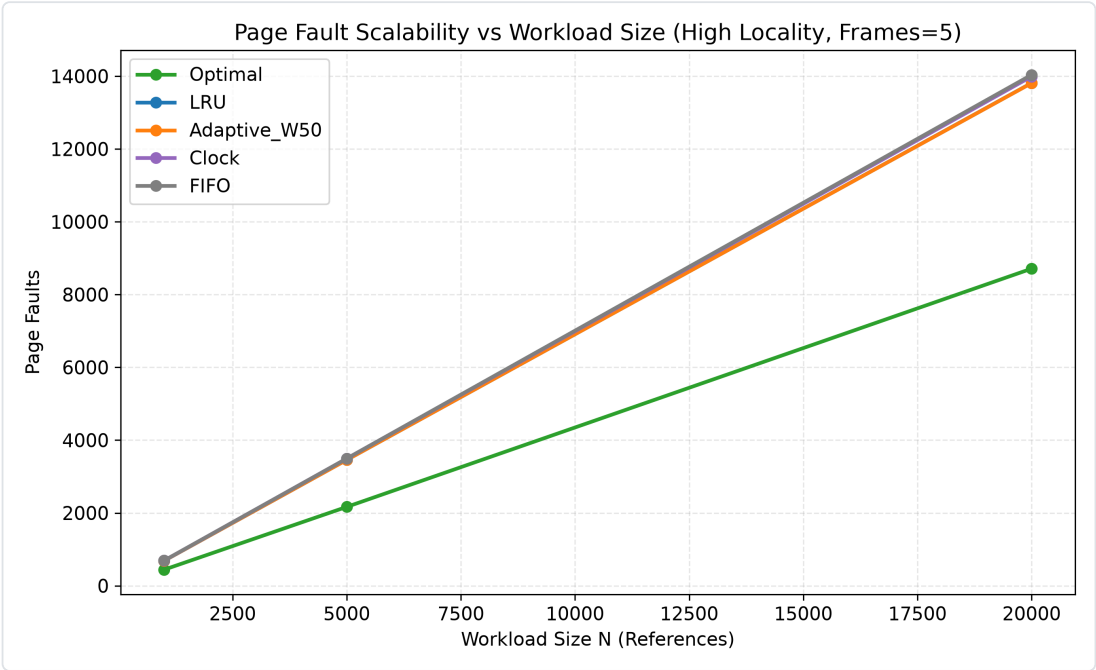


Figure 7: Page Fault Scalability vs Workload Size (High Locality, Frames=5).

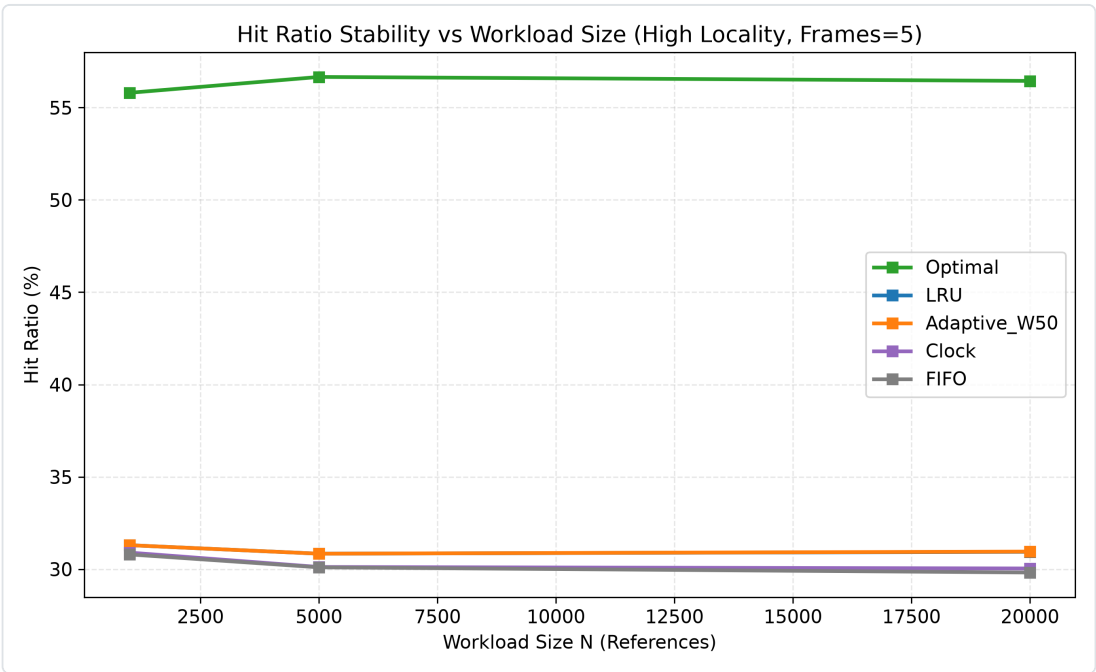


Figure 8: Hit Ratio Stability vs Workload Size (High Locality, Frames=5).

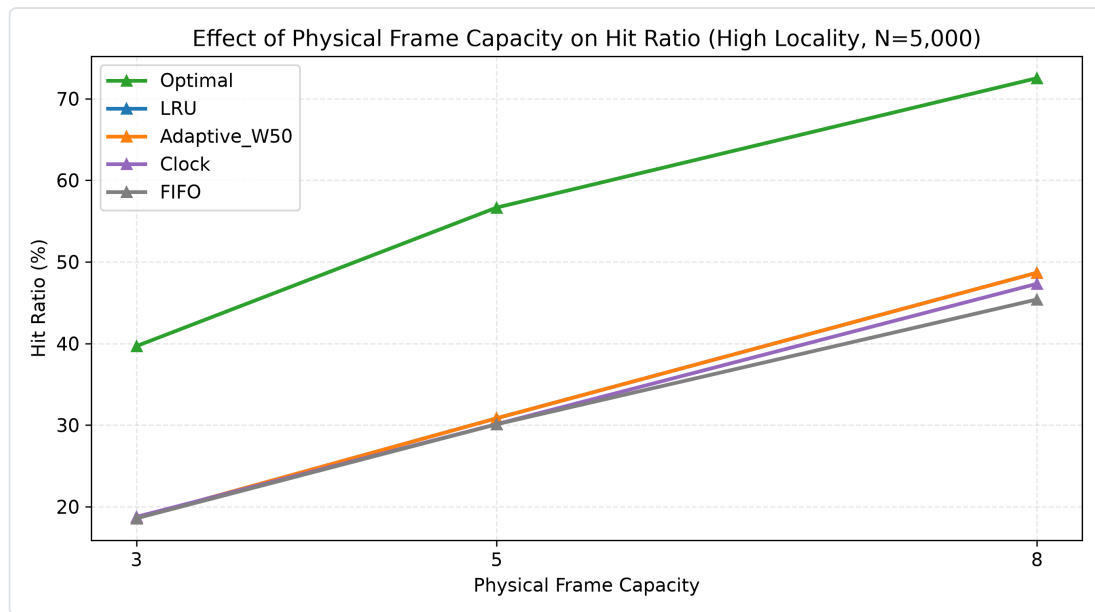


Figure 9: Effect of Frame Capacity on Hit Ratio (High Locality, N=5,000).